

目录

平台愿景与定位	3
整体架构设计	3
四层架构	3
多模态数据流	4
多模态 Agent 适配器体系	4
模态适配器矩阵	5
适配器接口规范	5
创作流水线系统	5
流水线声明格式	6
预置流水线模板	6
跨模态数据流转	6
提示词分层组合系统	7
风格系统 (STYLE.md)	8
核心包设计	8
Monorepo 结构	8
contracts 包 - 共享契约	9
media-engine 包 - 媒体引擎	9
modality-bridge 包 - 模态桥接	10
创作工作台设计	10
四面板布局	10
实时协作预览	11
SSE 多模态事件协议	11
插件系统设计	12
插件清单扩展	12

能力门控与安全	12
多模态质量审核系统	13
技术栈全景	14
与 Open Design 的架构对比	15
实施路线图	16
Phase 1 关键里程碑	16
总结	17

平台愿景与定位

OmniForge 是一个面向创作者的 AI Agent Studio 多模态创作平台，旨在将多种 AI 创作能力统一编排到一个流畅的创作工作流中。平台借鉴 Open Design 项目"Agent 无关、本地优先、插件化流水线"的架构哲学，将其从设计领域扩展到更广泛的多模态内容创作领域，支持文字创作、图像生成、视频合成和音乐制作四大核心创作模态。与传统的单一模态 AI 工具不同，OmniForge 不绑定任何特定的 AI 模型，而是通过统一的 Agent 适配器层兼容所有主流 AI 服务和本地模型，使创作者可以在一个统一的工作空间中自由组合不同模态的创作能力。

平台的核心价值主张是"创作流水线"（Creative Pipeline）——创作者可以定义从文字脚本到图像分镜、从音乐节奏到视频剪辑的端到端创作流程，由 AI Agent 自动执行各环节的创作任务，同时保持创作者对每个环节的细粒度控制。这种设计将"AI 辅助创作"从简单的单步生成提升为"多步编排 + 人机协同"的专业创作模式，使 AI 真正成为创作团队中的协作伙伴而非替代者。

属性	值
平台名称	OmniForge - AI Agent Studio
定位	多模态 AI 创作编排平台
架构模式	pnpm Monorepo + Agent 无关设计
支持模态	文字 / 图像 / 视频 / 音乐
核心理念	Agent 无关、本地优先、插件化流水线
运行模式	本地 CLI + 云端 API (BYOK) + 混合模式
技术栈	Node 24 + TypeScript + React 18 + Electron 41

表 1: 平台基本信息

整体架构设计

四层架构

借鉴 Open Design 的三层架构但进行扩展，OmniForge 采用四层架构以适应多模态创作的复杂性。新增的"创作引擎层"专门处理多模态内容的渲染、合成和转码，将媒体处理逻辑与业务编排逻辑分离，使得守护进程层可以专注于 Agent 编排和流式通信，而创作引擎层负责将

Agent 输出转化为最终的可消费媒体产物。

层级	技术栈	核心职责
创作工作台层	Next.js 16 + React 18 + Tailwind CSS 4	多轨编辑器、预览画布、时间轴、素材库、聊天交互
守护进程层	Express 5 + SQLite + Redis	Agent 生命周期管理、SSE 流式转发、插件引擎、MCP 服务器
创作引擎层	FFmpeg + Sharp + Web Audio API	媒体渲染、格式转码、实时合成、帧处理、音频混音
Agent 执行层	多模态 Agent 适配器	文字生成、图像生成、视频合成、音乐创作

表 2：四层架构职责划分

多模态数据流

OmniForge 的数据流围绕"创作项目"（Project）这一核心实体展开。每个创作项目包含一个或多个"创作轨道"（Track），每个轨道对应一种创作模态（文字轨、图像轨、视频轨、音乐轨），轨道之间通过"同步锚点"（Sync Anchor）建立时间对齐关系。用户在创作工作台发起创作请求后，守护进程根据当前活跃的创作流水线，依次或并行地调度相应的 Agent 执行创作任务。Agent 的输出通过 SSE 流式返回到前端，文字内容实时渲染到编辑器中，图像/视频/音频内容则通过创作引擎层进行实时处理和预览。

与 Open Design 单一 HTML artifact 流式渲染不同，OmniForge 需要处理多种媒体类型的流式传输和渐进式渲染。文字内容采用 SSE 文本增量流；图像采用 Base64 分块传输 + 渐进式 JPEG 解码；视频采用 HLS/DASH 分段流 + 关键帧预览；音频采用 Web Audio API 流式解码 + 波形预览。这种多协议流式架构确保了所有模态的内容都能实现"边生成边预览"的即时创作体验。

多模态 Agent 适配器体系

借鉴 Open Design 的 Agent 适配器模式，OmniForge 将适配器从"CLI 工具适配"扩展为"模态 + 提供者"二维适配。每个适配器不仅需要处理特定 AI 服务的 API 协议，还需要理解其输出模态的格式规范，将异构的 API 响应统一为平台内部的标准化创作事件。这种二维适配设计使得添加新的 AI 服务（如新的图像生成 API）或新的创作模态（如 3D 模型）都只需编写一个适配器文件即可。

模态适配器矩阵

创作模态	支持的服务	输出格式	流式协议
文字创作	OpenAI GPT / Claude / Gemini / DeepSeek / 本地 Ollama	Markdown / HTML / Plain Text	SSE text_delta
图像生成	DALL-E 3 / Stable Diffusion / Midjourney / Flux / ComfyUI	PNG / JPEG / WebP	Base64 分块 + 渐进 JPEG
视频合成	Runway / Pika / Kling / Sora / Seedance / Veo	MP4 / WebM	HLS 分段 + 关键帧预览
音乐创作	Suno / Udio / ElevenLabs / MusicGen / AudioCraft	MP3 / WAV / MIDI	Web Audio 流式解码

表 3: 多模态 Agent 适配器矩阵

适配器接口规范

每个适配器必须实现以下统一接口：`resolve()` 方法用于检测本地是否安装了对应的 CLI 工具或验证 API Key 的有效性；`buildArgs()` 方法根据创作请求构建服务特定的参数；`parseStream()` 方法将服务的原始输出流解析为标准化的创作事件；`diagnose()` 方法提供认证和连接的诊断信息。这种接口规范借鉴了 Open Design 的适配器模式，但增加了模态感知层，使得守护进程可以根据创作请求的模态类型自动路由到正确的适配器集合。

架构优势：二维适配（模态 x 提供者）使得平台可以同时支持同一模态下的多个 AI 服务，创作者可以根据成本、质量和速度的权衡自由选择。添加新服务只需一个适配器文件，添加新模态只需扩展事件协议。

创作流水线系统

创作流水线是 OmniForge 最核心的架构创新，借鉴并扩展了 Open Design 的插件流水线模式。在 Open Design 中，流水线是"发现 - 方向 - 评审 - 交付"的设计迭代流程；在 OmniForge 中，流水线被泛化为"多步骤、多模态、可循环"的创作编排引擎，支持跨模态的创作步骤串联、并行执行和条件分支。

流水线声明格式

每条创作流水线通过 pipeline.yaml 文件声明，包含阶段（stages）、步骤（steps）和原子操作（atoms）三个层次。阶段定义了创作流程的宏观阶段（如"创意构思"、"内容生成"、"质量审核"、"成品输出"），步骤定义了阶段内的具体操作（如"生成文字脚本"、"根据脚本生成分镜图"、"为分镜配乐"），原子操作则是不可再分的最小创作单元（如"调用图像生成 API"、"执行风格迁移"、"混音合成"）。步骤之间通过 input/output 契约连接，上游步骤的输出自动成为下游步骤的输入，实现跨模态的数据流转。

预置流水线模板

流水线名称	创作场景	阶段流程	涉及模态
故事板创作	短视频/动画脚本	构思 - 脚本 - 分镜 - 评审 - 输出	文字 + 图像
音乐视频	MV/配乐视频	歌词 - 旋律 - 画面 - 同步 - 合成	文字 + 音乐 + 视频
图文专栏	公众号/博客	选题 - 大纲 - 撰文 - 配图 - 排版	文字 + 图像
播客制作	音频播客	选题 - 脚本 - 录音 - 后期 - 发布	文字 + 音乐
品牌素材	营销物料	策略 - 文案 - 视觉 - 延展 - 交付	文字 + 图像 + 视频
交互课件	教育培训	目标 - 脚本 - 课件 - 动画 - 评测	文字 + 图像 + 视频 + 音乐

表 4：预置创作流水线模板

跨模态数据流转

流水线的核心挑战在于跨模态的数据流转。文字脚本的情节描述需要转化为图像生成的提示词，音乐的情绪标签需要映射到视频的色调风格。OmniForge 通过"模态桥接器"（Modality Bridge）解决这一问题：每个桥接器定义了从源模态到目标模态的转换规则，例如 TextToImageBridge 将文字描述提取为图像提示词（包含主体、风格、构图、色彩等维度），TextToMusicBridge 将文字的情感和节奏信息映射为音乐生成的参数（速度、调性、乐器、情绪等）。桥接器本身也是可插拔的，用户可以通过插件注册自定义的跨模态转换逻辑。

提示词分层组合系统

借鉴 Open Design 的 14 层提示词组合架构，OmniForge 将提示词系统扩展为 18 层，增加了多模态创作特有的约束层。系统提示词的组装遵循严格的优先级顺序，高层可以覆盖低层的默认行为，确保了提示词的一致性和可预测性。新增的层次主要包括：创作模态约束层（定义当前步骤的输出模态和格式要求）、风格系统层（类似设计系统，但扩展到视觉、听觉和叙事风格）、跨模态上下文层（上游步骤的输出作为下游步骤的输入约束）和质量门控层（定义各模态的质量检查标准）。

优先级	层级名称	说明
1	API 模式覆盖	BYOK 模式下的系统提示词覆盖
2	创作模式覆盖	创作/编辑/审核等模式的提示词差异
3	创作模态约束	当前步骤的输出模态和格式要求（新增）
4	发现与哲学层	Agent 能力发现和创作哲学声明
5	基础创作者章程	核心创作原则和伦理约束
6	个人记忆	从历史创作中自动提取的偏好
7	用户级指令	创作者全局偏好和风格倾向
8	项目级指令	当前创作项目的特定指令
9	风格系统 (STYLE.md)	视觉/听觉/叙事风格令牌（扩展）
10	创作技能 (SKILL.md)	特定创作技能的指令和示例
11	跨模态上下文	上游步骤输出作为输入约束（新增）
12	插件块	活跃插件的上下文注入
13	活跃流水线阶段	当前创作阶段的指令
14	项目元数据	项目类型、目标平台、受众等
15	质量门控	各模态的质量检查标准（新增）
16	品牌/版权约束	品牌规范和版权合规要求（新增）
17	平台输出规范	目标平台的格式和尺寸规范
18	媒体生成契约	分辨率、时长、码率等技术约束

表 5: 18 层提示词组合优先级

风格系统 (STYLE.md)

风格系统是 Open Design 设计系统概念的多模态扩展。每套风格系统由一个 STYLE.md 文件定义，包含视觉风格（色彩方案、字体规范、构图规则、滤镜预设）、听觉风格（乐器偏好、节奏范围、情感调性、混响参数）、叙事风格（人称视角、叙事节奏、语言风格、情感曲线）三大维度。Agent 在执行创作任务时会将 STYLE.md 中的风格规范作为权威约束，确保跨模态的创作产出在风格上保持一致。例如，一套"赛博朋克"风格系统可能定义霓虹色调的视觉风格、合成器主导的电子音乐风格和冷峻简练的叙事风格，确保文字、图像、视频和音乐在同一个创作项目中呈现统一的赛博朋克美学。

核心包设计

Monorepo 结构

OmniForge 采用与 Open Design 相同的 pnpm workspace monorepo 结构，但根据多模态创作的需求进行了包的重新组织和扩展。核心设计原则保持不变：contracts 包作为架构缝合点、plugin-runtime 为纯逻辑引擎、sidecar 提供进程间通信。新增的包主要围绕多模态媒体处理和创作编排两个维度展开。

包名	类型	核心职责
apps/studio	应用	Next.js 16 创作工作台 - 多轨编辑器、预览画布、素材库
apps/daemon	应用	Express 5 后端 - Agent 编排、SSE 流式、插件引擎
apps/desktop	应用	Electron 41 桌面壳 - 原生媒体处理、文件系统访问
apps/landing	应用	Astro 6 营销站点
packages/contracts	包	共享契约 - API 类型、Zod 模式、SSE 协议、提示词组合
packages/plugin-runtime	包	插件引擎 - 流水线解析、原子注册、模态桥接
packages/media-engine	包	媒体引擎 - FFmpeg 封装、Sharp 图像处理、音频混音

packages/modality-bridge	包	模态桥接 - 跨模态数据转换、提示词映射
packages/style-system	包	风格系统 - STYLE.md 解析、风格令牌、一致性校验
packages/timeline-sync	包	时间轴同步 - 多轨对齐、同步锚点、播放控制
packages/sidecar	包	进程通信 - IPC 协议、端口分配、Sidecar 运行时
packages/host	包	桌面桥接 - 原生能力暴露、文件系统、系统通知
packages/components	包	UI 组件库 - 编辑器组件、预览器、时间轴控件
plugins/_official	生态	官方插件 - 流水线模板、原子操作、风格系统
styles/	生态	风格系统库 - 赛博朋克、极简、复古等预设风格
skills/	生态	创作技能库 - SKILL.md 技能定义

表 6: 核心包结构

contracts 包 - 共享契约

contracts 包是 OmniForge 架构的基石，定义了创作工作台与守护进程之间的所有接口契约。与 Open Design 的 contracts 包类似，它采用纯 TypeScript + Zod 实现，零 Node.js 依赖，确保前后端的双向可消费性。关键扩展包括：多模态创作事件类型（TextDeltaEvent、ImageChunkEvent、VideoSegmentEvent、AudioFrameEvent）、创作轨道数据模型（Track、Clip、SyncAnchor）、流水线声明模式（PipelineManifest 的 Zod 模式）、风格系统令牌模式（StyleToken、VisualStyle、AudioStyle、NarrativeStyle）以及模态桥接器接口定义。composeSystemPrompt 函数从 14 层扩展为 18 层，新增的模态约束层和跨模态上下文层确保了多步骤创作流程中的提示词一致性。

media-engine 包 - 媒体引擎

这是 OmniForge 相对于 Open Design 新增的核心包，专门处理多模态媒体的渲染、合成和转码。该包封装了 FFmpeg（视频/音频处理）、Sharp（图像处理）和 Web Audio API（音频合成）三大媒体处理引擎，提供统一的 MediaEngine 接口。关键能力包括：图像的裁剪、缩放、滤镜和格式转换；视频的剪辑、拼接、转码和关键帧提取；音频的混音、淡入淡出、节拍检测和波形生成；以及多轨媒体的同步合成（将文字、图像、视频和音乐合成为最终的作品）。媒体引擎同时支持本地处理（通过 FFmpeg/Sharp）和云端处理（通过集成

Cloudinary/Mux 等服务），由守护进程根据项目配置自动路由。

modality-bridge 包 - 模态桥接

模态桥接器是跨模态创作的关键中间件。每个桥接器定义了从源模态到目标模态的数据转换规则，将上游步骤的输出自动转化为下游步骤可消费的输入。例如：TextToImageBridge 从文字描述中提取主体、风格、构图和色彩四个维度，组合为结构化的图像生成提示词；TextToMusicBridge 分析文字的情感极性、节奏感和场景氛围，映射为音乐生成的 tempo、key、mood 和 instruments 参数；ImageToVideoBridge 从静态图像推导镜头运动、景深变化和转场效果，生成视频合成的关键帧序列。桥接器通过插件注册，用户可以自定义或覆盖默认的桥接逻辑。

创作工作台设计

创作工作台是 OmniForge 的用户交互核心，借鉴 Open Design 的 Studio 界面但进行了大幅度的多模态扩展。工作台采用"画布 + 时间轴 + 素材库 + 对话"四面板布局，支持创作者在不同模态之间无缝切换和协同创作。

四面板布局

- **创作画布 (Canvas)**：中央区域，根据当前选中的轨道显示对应模态的预览——文字轨显示富文本编辑器，图像轨显示图片预览和编辑工具，视频轨显示视频播放器和帧浏览器，音乐轨显示波形图和频谱分析器。画布支持"分屏预览"模式，同时显示多个轨道的产出。
- **时间轴 (Timeline)**：底部区域，多轨道水平时间轴，类似视频编辑软件的轨道布局。每个轨道对应一种创作模态，通过同步锚点对齐不同轨道的时间关系。支持拖拽调整片段位置、缩放时间刻度、标记关键帧。
- **素材库 (Library)**：左侧面板，管理项目中的所有创作素材——Agent 生成的文字片段、图像、视频片段和音频片段。支持按模态筛选、按时间排序、标签分类和收藏标记。素材可以直接拖拽到时间轴或画布中使用。
- **创作对话 (Chat)**：右侧面板，与 AI Agent 的交互窗口。支持自然语言指令、流水线步骤控制、参数微调和创作反馈。Agent 的回复不仅包含文字，还可以直接在对话中嵌入生成的图像、音频片段和视频预览。

实时协作预览

借鉴 Open Design 的 artifact 流式渲染，OmniForge 实现了多模态的实时协作预览。文字内容通过 SSE text_delta 事件增量渲染到 Lexical 富文本编辑器中；图像内容通过 Base64 分块传输实现渐进式 JPEG 解码，用户可以看到图像从模糊到清晰的逐步呈现过程；视频内容通过 HLS 分段流实现边下载边播放，同时提取关键帧生成缩略图时间轴；音频内容通过 Web Audio API 的 AudioWorklet 实现流式解码和实时波形渲染。四种模态的预览在同一工作台并行运行，由时间轴同步系统确保多轨内容的播放对齐。

SSE 多模态事件协议

OmniForge 的 SSE 协议在 Open Design 的基础上进行了多模态扩展。原始协议仅支持文字类型的事件（text_delta、thinking_delta 等），新协议增加了图像、视频和音频三类媒体事件的规范。所有事件都遵循统一的信封格式，包含事件类型、轨道 ID、时间戳和载荷数据，确保前端可以用同一套事件分发器处理所有模态的内容。

事件类型	载荷格式	说明
text_delta	{ trackId, delta, metadata }	文字增量，包含格式标记
image_chunk	{ trackId, chunkIndex, base64, progress }	图像分块，支持渐进式渲染
image_complete	{ trackId, url, width, height, format }	图像完成，提供完整 URL
video_segment	{ trackId, segmentUrl, duration, keyframes }	视频分段，HLS/DASH 格式
video_complete	{ trackId, url, duration, resolution }	视频完成，提供完整 URL
audio_frame	{ trackId, pcmData, sampleRate, channels }	音频帧，PCM 原始数据
audio_complete	{ trackId, url, duration, waveform }	音频完成，附带波形数据
bridge_output	{ sourceTrackId, targetTrackId, mapping }	模态桥接输出，跨轨数据映射
pipeline_stage	{ stage, step, status, progress }	流水线阶段状态更新

quality_review	{ trackId, scores, suggestions }	质量审核结果和改进建议
----------------	----------------------------------	-------------

表 7: SSE 多模态事件类型

插件系统设计

OmniForge 的插件系统在 Open Design"插件即文件系统"的基础上，增加了对多模态创作原子的支持。插件仍然是包含 SKILL.md + omni-forge.json 清单的文件夹，但清单格式扩展了模态声明、流水线原子注册和模态桥接器注册三个新的配置段。这种设计保持了极低的插件创建门槛——只需一个 SKILL.md 文件即可定义最简单的创作技能，而复杂的跨模态流水线则通过清单文件进行声明式编排。

插件清单扩展

配置段	说明	示例
of.modality	声明插件支持的创作模态	text, image, video, audio
of.pipeline.atoms	注册流水线原子操作	image-style-transfer, audio-beat-match
of.bridge	注册模态桥接器	text-to-image-prompt, script-to-storyboard
of.capabilities	能力声明（权限控制）	prompt:inject, media:read, media:write, fs:read
of.genui	GenUI 界面定义	参数调节面板、风格选择器、预览卡片
of.inputs	输入定义	文字脚本、参考图像、音频片段

表 8: 插件清单扩展字段

能力门控与安全

借鉴 Open Design 的能力门控机制，OmniForge 将权限粒度从"文件系统操作"扩展到"媒体操作"。新增的能力声明包括：media:read（读取媒体素材）、media:write（生成/修改媒体素材）、media:render（调用媒体引擎进行渲染合成）、network:api（调用外部 API 服务）和 modality:bridge（执行跨模态转换）。受限安装默认仅获得 prompt:inject 权限，需

要媒体操作的插件必须在安装时声明并获得用户授权。这种分级权限设计在保持插件生态开放性的同时，有效防止了恶意插件对用户媒体素材的未授权访问。

多模态质量审核系统

借鉴 Open Design 的 Critique Theater 多专家评审机制，OmniForge 将评审维度从设计领域扩展到多模态创作领域。系统模拟 7 位评审专家对创作产出进行多维度评估，每个专家对应一个特定的审核维度。与 Open Design 的 5 位设计评审专家不同，OmniForge 的 7 位专家覆盖了多模态创作的完整质量谱系，确保文字的叙事质量、图像的视觉质量、视频的动态质量、音频的声学质量以及跨模态的整体一致性都得到充分评估。

评审专家	审核维度	评分指标
叙事架构师	文字内容的叙事逻辑和结构	情节连贯性、角色一致性、节奏把控、主题深度
视觉总监	图像和视频的视觉质量	构图质量、色彩和谐、细节丰富度、风格一致性
音乐总监	音频内容的声音质量	旋律质量、节奏稳定性、混音平衡、情感匹配
品牌守护者	与风格系统/品牌规范的一致性	色彩合规、字体规范、风格调性、品牌元素
无障碍专家	内容的可访问性	文字可读性、图像对比度、音频清晰度、字幕完整
跨模态协调员	多模态之间的整体一致性	视听同步、情感统一、节奏协调、叙事衔接
技术质检员	技术规格的合规性	分辨率/码率达标、格式兼容、无损坏/伪影

表 9：多模态质量审核专家体系

质量审核系统同样采用多轮迭代机制：首轮审核给出各维度的评分和改进建议，Agent 根据建议自动修改创作产出，然后进入下一轮审核，直到所有维度的评分都达到配置的阈值或达到最大迭代轮次。审核过程通过 SSE 协议实时推送到前端，创作者可以观看审核直播、查看历史审核记录，并在任意时刻介入修改。系统继承 Open Design 的滚动式灰度发布策略，通过 16 个阶段的渐进式启用控制质量审核功能的上线节奏。

技术栈全景

OmniForge 的技术栈在 Open Design 的基础上增加了多模态媒体处理和创作编排所需的工具链。核心原则保持不变：TypeScript 全栈、esbuild 快速打包、Vitest 单元测试、Playwright E2E 测试。新增的工具主要围绕媒体处理（FFmpeg、Sharp）、音频合成（Web Audio API、Tone.js）和实时协作（CRDT、WebSocket）。

类别	技术	用途
后端框架	Express 5	HTTP 服务器 + API 路由 + SSE 流式
数据库	SQLite + Redis	本地存储 + 缓存/会话/任务队列
媒体处理	FFmpeg + Sharp	视频/音频处理 + 图像处理
音频合成	Tone.js + Web Audio API	浏览器端音频合成和混音
实时协作	Yjs (CRDT) + WebSocket	多人协作编辑和状态同步
类型校验	Zod	运行时类型验证
前端框架	Next.js 16 (App Router)	React 框架
富文本编辑	Lexical	文字创作编辑器
时间轴	自定义 Canvas 组件	多轨时间轴编辑器
波形渲染	WaveSurfer.js	音频波形可视化
视频播放	Video.js + HLS.js	视频流式播放
桌面端	Electron 41	原生桌面壳 + 本地媒体处理
构建工具	esbuild + Turbopack	快速打包
测试	Vitest + Playwright	单元测试 + E2E 测试
监控	PostHog + Prometheus	分析 + 指标

表 10: 技术栈全景

与 Open Design 的架构对比

OmniForge 在架构层面深度借鉴了 Open Design 的核心模式，但根据多模态创作的需求进行了关键性的扩展和调整。以下是两个项目在核心架构维度上的详细对比，清晰地展示了哪些模式被直接复用、哪些被扩展、哪些被重新设计。

架构维度	Open Design	OmniForge	演进策略
架构层次	三层（前端/守护进程/Agent）	四层（+创作引擎层）	扩展：媒体处理独立为层
Agent 适配器	CLI 工具一维适配	模态 x 提供者二维适配	扩展：增加模态感知维度
提示词组合	14 层	18 层	扩展：增加模态约束/桥接/质量层
设计/风格系统	DESIGN.md（视觉）	STYLE.md（视觉+听觉+叙事）	扩展：多模态风格统一
SSE 协议	文字事件为主	多模态事件（图/视频/音频）	扩展：增加媒体事件类型
插件系统	SKILL.md + open-design.json	SKILL.md + omni-forge.json	扩展：模态声明/桥接器注册
质量审核	5 专家（设计领域）	7 专家（多模态领域）	扩展：增加跨模态协调维度
契约层	contracts（纯 TS+Zod）	contracts（纯 TS+Zod）	复用：零变更架构策略
Sidecar IPC	Unix Socket / Named Pipe	同左	复用：进程通信机制
本地优先	localhost + SQLite	localhost + SQLite + Redis	扩展：增加任务队列和缓存
Artifact 渲染	HTML iframe 沙箱	多模态预览器矩阵	重新设计：多模态并行渲染
编辑模式	对话式单流	对话 + 多轨时间轴	重新设计：专业创作 workflow

表 11: 与 Open Design 的架构对比

实施路线图

OmniForge 的开发采用渐进式交付策略，每个阶段交付一个可用的最小产品，逐步扩展创作模态和流水线能力。这种策略借鉴了 Open Design 从 0.1 到 0.10 的迭代经验，确保每个版本都能为用户提供实际价值。

阶段	版本	核心交付	创作能力
Phase 1	v0.1	Monorepo 骨架 + contracts 包 + 文字 Agent 适配器	文字创作（单模态）
Phase 2	v0.3	图像 Agent 适配器 + 媒体引擎 + 画布预览	文字 + 图像（双模态）
Phase 3	v0.5	创作流水线 + 模态桥接器 + 风格系统	跨模态编排
Phase 4	v0.7	视频 + 音频适配器 + 时间轴编辑器	四模态完整
Phase 5	v0.9	质量审核 + 插件市场 + 桌面端	生态完善
Phase 6	v1.0	多人协作 + 部署服务 + 性能优化	生产就绪

表 12：实施路线图

Phase 1 关键里程碑

Phase 1 是整个项目的基础，需要在第一个月内交付可运行的最小产品。关键里程碑包括：搭建 pnpm monorepo 工作区，配置 TypeScript、esbuild 和 Vitest；实现 contracts 包的核心类型定义，包括 ChatRequest、CreativeEvent、Track、Project 等基础数据模型；实现 daemon 的基础 Express 服务器，支持 SSE 流式通信和 SQLite 持久化；实现文字模态的 5 个 Agent 适配器（OpenAI、Claude、Gemini、DeepSeek、Ollama）；实现 studio 的基础聊天界面和文字预览编辑器。Phase 1 完成后，用户应该能够通过对话方式与 AI Agent 交互进行文字创作，并在编辑器中实时看到生成的内容。这验证了核心的 Agent 编排和 SSE 流式渲染架构。

总结

OmniForge 的架构设计深度借鉴了 Open Design 的核心架构模式——Agent 无关设计、分层提示词组合、声明式插件流水线、契约层为锚点、SSE 流式通信和本地优先安全设计，并将这些模式从单一的设计领域扩展到了文字、图像、视频、音乐四大创作模态。核心的架构创新包括：四层架构（新增创作引擎层）、二维 Agent 适配器（模态 x 提供者）、18 层提示词组合（新增模态约束/跨模态上下文/质量门控）、模态桥接器（跨模态数据流转）、风格系统（多模态风格统一）和多模态质量审核（7 专家评审体系）。

这些扩展并非简单的功能堆叠，而是在保持 Open Design 架构优雅性的前提下，针对多模态创作的本质挑战——跨模态数据流转、多轨时间同步、异构媒体渲染和跨模态风格一致性——进行的结构性创新。contracts 包的零 Node 依赖策略确保了前后端的松耦合，plugin-runtime 的纯逻辑设计使得插件可以在任何环境中运行，而 Sidecar IPC 的进程隔离则为媒体处理的安全性和稳定性提供了保障。这些从 Open Design 继承的架构基因，使得 OmniForge 在面对多模态创作的复杂性时，依然保持了架构的清晰性和可扩展性。

展望未来，OmniForge 的架构预留了多个扩展方向：3D 模型创作模态的接入（通过扩展 modality-bridge 和 media-engine）、实时协作编辑（通过 Yjs CRDT 实现多人同时编辑同一创作项目）、AI 角色系统（为不同创作任务配备专门的 AI 角色）、以及社区创作市场的建设（创作者可以分享和交易流水线模板、风格系统和创作技能）。这些扩展方向都可以在当前架构的框架内实现，无需进行大规模的架构改造，这正是良好架构设计的证明。